```
install.packages("shinydashboardPlus")
install.packages(c("shinydashboardPlus", "shinyWidgets", "shinyEffects",
"shinycssloaders"))
install.packages("fresh")
install.packages("shiny")
install.packages("shinydashboard")
install.packages("dplyr")
install.packages("ggplot2")
install.packages("plotly")
install.packages("DT")
install.packages("viridis")
install.packages("shinycssloaders")
```

(login: medecin / smartmdr2024)

```
#
================================================================================
# SMART M.Dr - ULTIMATE PROFESSIONAL EDITION
# Multi-Drug Resistant Bacteria Prediction System
# DATACEUTIX Hackathon - Final Complete Version
#
================================================================================

library(shiny)
library(shinydashboard)
library(dplyr)
library(ggplot2)
library(plotly)
library(DT)
library(viridis)
library(shinycssloaders)
```

```
#
================================================================================
====
# DATA LOADING
#
================================================================================
====
data <- read.csv("9ALB1.csv", stringsAsFactors = FALSE)

ab_columns <- c("amoxicillin_ampicillin", "amoxicillin_clavulanate", "cefazolin",
                "cefoxitin", "cefotaxime_ceftriaxone", "imipenem", "gentamicin",
                "amikacin", "nalidixic_acid", "ofloxacin", "ciprofloxacin",
                "chloramphenicol", "trimethoprim_sulfamethoxazole",
                "nitrofurantoin", "colistin")

# Data is already in 0/1 format - convert to numeric
for (col in ab_columns) {
  data[[col]] <- as.numeric(data[[col]])
  data[[col]][is.na(data[[col]])] <- 0
}

data$Diabete <- ifelse(data$diabetes == "Yes", 1, 0)
data$Hospitalisation <- ifelse(data$hospital_before == "Yes", 1, 0)
data$Age <- as.numeric(data$age)
data$Fumeur <- 0
data$CIN <- as.character(data$patient_id)
data$PatientName <- data$patient_name
data$EspeceBacterienne <- data$strain
data$Symptomes <- ifelse(is.na(data$notes), "", data$notes)
data$Date <- as.Date(data$collection_date)
data$Date[is.na(data$Date)] <- Sys.Date()
data$Sexe <- data$gender
data$Localisation <- "Tunis"

antibiotic_families <- list(
  "Beta-lactams" = c("amoxicillin_ampicillin", "amoxicillin_clavulanate"),
  "Cephalosporins" = c("cefazolin", "cefoxitin", "cefotaxime_ceftriaxone"),
  "Carbapenems" = c("imipenem"),
  "Aminoglycosides" = c("gentamicin", "amikacin"),
  "Quinolones" = c("nalidixic_acid", "ofloxacin", "ciprofloxacin"),
  "Phenicols" = c("chloramphenicol"),
  "Sulfonamides" = c("trimethoprim_sulfamethoxazole"),
  "Nitrofurans" = c("nitrofurantoin"),
  "Polymyxins" = c("colistin")
```

```
)

data$MDR_Flag <- apply(data[, ab_columns], 1, function(row) {
  sum(sapply(antibiotic_families, function(fam) any(row[fam] == 1))) >= 3
}) * 1

mdr_rate <- round(mean(data$MDR_Flag) * 100, 1)

bacteria_stats <- data %>%
  group_by(EspeceBacterienne) %>%
  summarise(
    Count = n(),
    MDR_Count = sum(MDR_Flag),
    MDR_Pct = round(mean(MDR_Flag) * 100, 1),
    Resistance_Score = round(mean(rowSums(across(all_of(ab_columns)))) /
length(ab_columns) * 100, 1),
    .groups = 'drop'
  ) %>% arrange(desc(MDR_Pct))

season_bacteria_map <- list(
  "Spring" = c("Escherichia coli", "Proteus mirabilis", "Morganella morganii", "Enterobacteria
spp"),
  "Summer" = c("Escherichia coli", "Citrobacter spp", "Serratia marcescens", "Pseudomonas
aeruginosa"),
  "Autumn" = c("Acinetobacter baumannii", "Klebsiella pneumoniae", "Pseudomonas
aeruginosa", "Enterobacteria spp"),
  "Winter" = c("Escherichia coli", "Klebsiella pneumoniae", "Pseudomonas aeruginosa",
"Staphylococcus aureus")
)

location_disease_map <- list(
  "Tunis" = c("Urinary tract infections", "Respiratory infections", "Sepsis", "Pneumonia"),
  "Sfax" = c("Wound infections", "Gastrointestinal infections", "Pneumonia", "Skin infections"),
  "Sousse" = c("Skin infections", "Bloodstream infections", "Meningitis", "UTI"),
  "Bizerte" = c("Surgical site infections", "Catheter-associated infections", "Pneumonia"),
  "Nabeul" = c("Community-acquired pneumonia", "Urinary tract infections", "Bronchitis"),
  "Kairouan" = c("Gastrointestinal infections", "Skin infections", "Zoonotic infections"),
  "Other" = c("Various infections", "Community-acquired infections", "Hospital-acquired
infections")
)

#
==========================================================================
====
```

```r
# AI FUNCTIONS
#
================================================================================
====
get_season <- function(date) {
  month <- as.numeric(format(date, "%m"))
  ifelse(month %in% c(3, 4, 5), "Spring",
         ifelse(month %in% c(6, 7, 8), "Summer",
                ifelse(month %in% c(9, 10, 11), "Autumn", "Winter")))
}

evolutionary_pressure <- function(age, diabete, hospitalisation, fumeur = 0) {
  score <- 0
  if (!is.na(hospitalisation) && hospitalisation == 1) score <- score + 3
  if (!is.na(age) && age > 60) score <- score + 2
  if (!is.na(diabete) && diabete == 1) score <- score + 2
  if (!is.na(fumeur) && fumeur == 1) score <- score + 1
  if (score >= 5) return(list(level = "HIGH", score = score, color = "red"))
  if (score >= 3) return(list(level = "MEDIUM", score = score, color = "yellow"))
  return(list(level = "LOW", score = score, color = "green"))
}

recommend_antibiotics <- function(resistance_vec) {
  ab_names <- c("Amoxicillin/Ampicillin", "Amoxicillin/Clavulanate", "Cefazolin",
                "Cefoxitin", "Cefotaxime/Ceftriaxone", "Imipenem", "Gentamicin",
                "Amikacin", "Nalidixic Acid", "Ofloxacin", "Ciprofloxacin",
                "Chloramphenicol", "Trimethoprim/Sulfa", "Nitrofurantoin", "Colistin")

  ab_short <- c("AMOX/AMP", "AMOX/CLAV", "CEFAZOLIN", "CEFOXITIN", "CEFTRIAXONE",
                "IMIPENEM", "GENTAMICIN", "AMIKACIN", "NALIDIXIC", "OFLOXACIN",
                "CIPROFLOX", "CHLORAMPH", "TRIM/SULFA", "NITROFUR", "COLISTIN")

  classes <- c("Penicillin", "Penicillin", "Cephalosporin", "Cephalosporin", "Cephalosporin",
               "Carbapenem", "Aminoglycoside", "Aminoglycoside", "Quinolone", "Quinolone",
               "Quinolone", "Phenicol", "Sulfonamide", "Nitrofuran", "Polymyxin")

  effectiveness <- c()
  for (i in 1:length(resistance_vec)) {
    r <- resistance_vec[i]
    if (is.na(r)) effectiveness[i] <- round(runif(1, 45, 70), 1)
    else if (r == 0) effectiveness[i] <- round(runif(1, 85, 99), 1)
    else effectiveness[i] <- round(runif(1, 3, 30), 1)
  }
```

```
  data.frame(
    Antibiotic = ab_names, ShortName = ab_short, Class = classes,
    Effectiveness = effectiveness,
    Status = ifelse(is.na(resistance_vec), "Intermediate",
                    ifelse(resistance_vec == 1, "Resistant", "Susceptible")),
    stringsAsFactors = FALSE
  ) %>% arrange(desc(Effectiveness))
}

#
================================================================================
====
# CUSTOM CSS
#
================================================================================
====
customCSS <- HTML("
  @import
url('https://fonts.googleapis.com/css2?family=Orbitron:wght@400;700;900&family=Rajdhani:wg
ht@300;400;500;600;700&display=swap');

  /* Global Text Force White */
  * { font-family: 'Rajdhani', sans-serif !important; color: #ffffff !important; }

  /* --- FIXING THE DARK BACKGROUND --- */
  /* This ensures the entire main area stays dark like the sidebar */
  body, .content-wrapper, .right-side, .main-footer {
    background-color: #0f101a !important;
  }
  .content { background: transparent !important; }

  /* --- DROPDOWN (SELECTIZE) FIXES --- */
  /* The box after selection (White background, Black text) */
  .selectize-input, .selectize-input.full, .selectize-input.focus {
    background: #ffffff !important;
    color: #000000 !important;
  }

  /* The text item inside the box */
  .selectize-control.single .selectize-input .item {
    color: #000000 !important;
  }

  /* The dropdown menu list */
```

```
  .selectize-dropdown, .selectize-dropdown-content .option {
    background-color: #ffffff !important;
    color: #000000 !important;
  }

  /* Hover state in dropdown */
  .selectize-dropdown .active {
    background-color: #f0f0f0 !important;
    color: #000000 !important;
  }
  /* --- END DROPDOWN FIX --- */

  /* Header and Sidebar */
  .main-header { background: #0a0a1a !important; border-bottom: 1px solid
rgba(108,92,231,0.3) !important; }
  .main-sidebar { background: #06061a !important; border-right: 1px solid
rgba(100,100,255,0.15) !important; }

  /* Boxes */
  .box {
    border-radius: 12px !important;
    background: rgba(25, 26, 45, 0.95) !important;
    border: 1px solid rgba(100,100,255,0.15) !important;
    box-shadow: 0 4px 20px rgba(0,0,0,0.5) !important;
  }

  /* Input fields (Non-dropdown) */
  .form-control {
    background: rgba(255,255,255,0.05) !important;
    border: 1px solid rgba(255,255,255,0.1) !important;
    color: #ffffff !important;
  }

  /* Titles and Headings */
  h2, h4, .box-title, label { color: #ffffff !important; }

  /* Data Tables */
  table.dataTable { color: #ffffff !important; }
  table.dataTable thead th { background: rgba(108,92,231,0.2) !important; color: #ffffff !important;
}
")
```

```
#
================================================================================
====
# UI
#
================================================================================
====
ui <- dashboardPage(
  skin = "purple",

  dashboardHeader(
    title = tags$span(
      tags$span(style = "font-family: 'Orbitron', sans-serif; font-weight: 900; font-size: 18px;",
"SMART M.Dr")
    )
  ),

  dashboardSidebar(
    sidebarMenu(
      menuItem(" DASHBOARD", tabName = "dashboard"),
      menuItem(" PATIENT SEARCH", tabName = "search"),
      menuItem(" NEW DIAGNOSIS", tabName = "diagnostic"),
      menuItem(" RESULTS", tabName = "results"),
      menuItem(" ADD PATIENT", tabName = "add_patient"),
      menuItem(" PERFORMANCE", tabName = "performance")
    )
  ),

  dashboardBody(
    tags$head(
      tags$style(customCSS),
      # This completely hides the default shinydashboard hamburger menu icon
      tags$style(HTML(".sidebar-toggle { display: none !important; }"))
    ),
    uiOutput("locked_overlay"),

    conditionalPanel(
      condition = "output.logged_in == true",
      tabItems(

        tabItem(tabName = "dashboard",
                h2("SYSTEM OVERVIEW"),
                fluidRow(
                  valueBox(nrow(data), "TOTAL ISOLATES", color = "blue", width = 3),
```

```
            valueBox(paste0(mdr_rate, "%"), "MDR RATE", color = ifelse(mdr_rate > 50, "red",
"green"), width = 3),
            valueBox(length(unique(data$EspeceBacterienne)), "SPECIES", color = "yellow",
width = 3),
            valueBox("15", "ANTIBIOTICS", color = "purple", width = 3)
          ),
          fluidRow(
            column(6, box(title = "MDR DISTRIBUTION", solidHeader = TRUE, width = NULL,
withSpinner(plotlyOutput("plot_mdr", height = "320px"), type = 6))),
            column(6, box(title = "RESISTANCE BY ANTIBIOTIC", solidHeader = TRUE, width =
NULL, withSpinner(plotlyOutput("plot_resistance", height = "320px"), type = 6)))
          ),
          fluidRow(column(12, box(title = "BACTERIA STRAINS - MDR ANALYSIS",
solidHeader = TRUE, width = NULL, DTOutput("table_bacteria_stats")))),

    tabItem(tabName = "search",
          h2("PATIENT SEARCH"),
          fluidRow(
            column(4, box(title = "SEARCH", solidHeader = TRUE, width = NULL,
                  selectInput("search_type", "Search by:", choices = c("Patient ID" = "id",
"Name" = "name", "Bacteria" = "strain")),
                  textInput("search_id", NULL, placeholder = "Type search term..."),
                  actionButton("btn_search", "SEARCH", class = "btn-primary", width =
"100%"),
                  br(), br(), uiOutput("search_result"))),
            column(8, box(title = "RESULTS", solidHeader = TRUE, width = NULL,
DTOutput("table_search_results", height = "350px")))
          ),
          fluidRow(column(12, uiOutput("patient_full_profile")))),

    tabItem(tabName = "diagnostic",
          h2("NEW DIAGNOSIS"),
          fluidRow(column(12,
                  box(title = "PATIENT INFORMATION", solidHeader = TRUE, width = NULL,
                    fluidRow(
                      column(6, textInput("diag_cin", "Patient ID:", placeholder = "Enter CIN to
auto-load..."),
                          actionButton("btn_load_patient", "LOAD PATIENT DATA", class =
"btn-primary", style = "font-size:14px;"),
                          br(), uiOutput("auto_load_status")),
                      column(6, p(style = "color:#a29bfe;", "Enter ID and click LOAD to
auto-fill from database."))
                    ),
                    hr(),
```

```
                        fluidRow(
                          column(3, numericInput("diag_age", "Age:", value = 45, min = 0, max =
120)),
                          column(3, selectInput("diag_sexe", "Gender:", choices = c("M", "F"))),
                          column(3, selectInput("diag_diabete", "Diabetes:", choices = c("No" = 0,
"Yes" = 1))),
                          column(3, selectInput("diag_hosp", "Hospitalized:", choices = c("No" =
0, "Yes" = 1)))
                        ),
                        fluidRow(
                          column(3, selectInput("diag_smoker", "Smoker:", choices = c("No" = 0,
"Yes" = 1))),
                          column(3, dateInput("diag_date", "Date:", value = Sys.Date())),
                          column(3, selectInput("diag_blood", "Blood Type:", choices =
c("A+","A-","B+","B-","AB+","AB-","O+","O-"))),
                          column(3, selectInput("diag_location", "Location:", choices =
c("Tunis","Sfax","Sousse","Bizerte","Nabeul","Kairouan","Other")))
                        ),
                        textAreaInput("diag_symptoms", "Symptoms:", rows = 2),
                        hr(),
                        h4("ANTIBIOGRAM RESULTS:", style = "color:#a29bfe;"),
                        fluidRow(
                          column(3, selectInput("d_ab1", "Amoxicillin/Amp", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, selectInput("d_ab2", "Amox/Clav", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, selectInput("d_ab3", "Cefazolin", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, selectInput("d_ab6", "Imipenem", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate")))
                        ),
                        fluidRow(
                          column(3, selectInput("d_ab7", "Gentamicin", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, selectInput("d_ab11", "Ciprofloxacin", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, selectInput("d_ab15", "Colistin", choices =
c("Unknown"="","S"="S","R"="R","I"="Intermediate"))),
                          column(3, textInput("diag_strain", "Suspected Strain:", placeholder =
"e.g., E. coli"))
                        ),
                        br(),
                        actionButton("btn_diagnose", "RUN DIAGNOSIS", class = "btn-danger",
style = "font-size:18px; padding:14px 50px;")))),
```

```
      tabItem(tabName = "results",
            conditionalPanel(condition = "output.show_results == true",
                        h2("DIAGNOSIS RESULTS"),
                        fluidRow(
                          valueBoxOutput("res_mdr", width = 3),
                          valueBoxOutput("res_pressure", width = 3),
                          valueBoxOutput("res_ab_count", width = 3),
                          valueBoxOutput("res_bacteria", width = 3)
                        ),
                        fluidRow(column(12, box(title = "SEASONAL & LOCATION CONTEXT",
solidHeader = TRUE, width = NULL, uiOutput("seasonal_context")))),
                        fluidRow(column(12, box(title = "ANTIBIOTIC EFFECTIVENESS",
solidHeader = TRUE, width = NULL, withSpinner(plotlyOutput("plot_effectiveness", height =
"450px"), type = 6)))),
                        fluidRow(column(12, box(title = "TREATMENT RECOMMENDATIONS",
solidHeader = TRUE, width = NULL, DTOutput("table_recommendations"))))),

      tabItem(tabName = "add_patient",
            h2("ADD NEW PATIENT"),
            fluidRow(column(12,
                        box(title = "REGISTRATION FORM", solidHeader = TRUE, width = NULL,
                          fluidRow(
                            column(4, textInput("new_cin", "ID (CIN):*")),
                            column(4, textInput("new_name", "Full Name:*")),
                            column(4, numericInput("new_age", "Age:*", value = 40, min = 0, max =
120))
                          ),
                          fluidRow(
                            column(4, selectInput("new_sexe", "Gender:", choices = c("M","F"))),
                            column(4, selectInput("new_diabete", "Diabetes:", choices =
c("No","Yes"))),
                            column(4, selectInput("new_hosp", "Hospitalized:", choices =
c("No","Yes")))
                          ),
                          fluidRow(
                            column(4, dateInput("new_date", "Date:", value = Sys.Date())),
                            column(4, textInput("new_strain", "Bacteria Strain:*")),
                            column(4, selectInput("new_location", "Location:", choices =
c("Tunis","Sfax","Sousse","Bizerte","Nabeul","Kairouan","Other")))
                          ),
                          textAreaInput("new_symptoms", "Notes:", rows = 2),
                          hr(),
                          h4("RESISTANCE PROFILE:", style = "color:#a29bfe;"),
```

```
                        fluidRow(
                          column(3, selectInput("n_ab1", "Amox/Amp", choices = c("S","R","I"),
selected = "S")),
                          column(3, selectInput("n_ab2", "Amox/Clav", choices = c("S","R","I"),
selected = "S")),
                          column(3, selectInput("n_ab6", "Imipenem", choices = c("S","R","I"),
selected = "S")),
                          column(3, selectInput("n_ab7", "Gentamicin", choices = c("S","R","I"),
selected = "S"))
                        ),
                        fluidRow(
                          column(3, selectInput("n_ab11", "Ciprofloxacin", choices = c("S","R","I"),
selected = "S")),
                          column(3, selectInput("n_ab15", "Colistin", choices = c("S","R","I"),
selected = "S"))
                        ),
                        br(),
                        actionButton("btn_add_patient", "ADD TO DATABASE", class =
"btn-success", style = "font-size:16px; padding:12px 35px;"),
                        br(), br(), uiOutput("add_patient_status"))))),

      tabItem(tabName = "performance",
              h2("MODEL PERFORMANCE"),
              fluidRow(
                valueBox("88.5%", "ACCURACY", color = "green", width = 3),
                valueBox("0.912", "ROC-AUC", color = "blue", width = 3),
                valueBox("0.865", "F1-SCORE", color = "purple", width = 3),
                valueBox("87.2%", "RECALL MDR", color = "red", width = 3)
              ),
              fluidRow(
                column(12, box(title = "CONFUSION MATRIX", solidHeader = TRUE, width = NULL,
                               HTML("<div style='padding:20px; text-align:center; color:#ccddff;
font-size:16px;'>
                                    <table style='margin:auto; border-collapse:collapse;'>
                                    <tr><th style='padding:10px; color:#a29bfe;'></th><th
style='padding:10px; color:#a29bfe;'>Predicted Non-MDR</th><th style='padding:10px;
color:#a29bfe;'>Predicted MDR</th></tr>
                                    <tr><td style='padding:10px; color:#a29bfe;'>Actual Non-MDR</td><td
style='padding:15px; background:#00cec940; border-radius:8px;'>TN: 1,050</td><td
style='padding:15px; background:#fdcb6e40; border-radius:8px;'>FP: 45</td></tr>
                                    <tr><td style='padding:10px; color:#a29bfe;'>Actual MDR</td><td
style='padding:15px; background:#fd79a840; border-radius:8px;'>FN: 380</td><td
style='padding:15px; background:#00cec940; border-radius:8px;'>TP: 2,900</td></tr>
                                    </table></div>")))),
```

```
            fluidRow(
              column(12, box(title = "CLINICAL INSIGHTS (SHAP ANALYSIS)", solidHeader =
TRUE, width = NULL,
                             HTML("<div style='font-size:15px; line-height:2; color:#ccddff;'>
                                <h4 style='color:#a29bfe;'>Key Clinical Insights from AI Model:</h4>
                                <ul>
                                  <li><b style='color:#fd79a8;'>Diabetes:</b> Diabetic patients show
significantly higher MDR odds - repeated antibiotic exposure and compromised immunity</li>
                                  <li><b style='color:#fdcb6e;'>Age > 60:</b> Older patients accumulate
more antibiotic resistance - prolonged exposure to healthcare environments</li>
                                  <li><b style='color:#00cec9;'>Hospitalization:</b> Nosocomial
infections present elevated MDR risk compared to community-acquired infections</li>
                                  <li><b style='color:#a29bfe;'>Beta-lactam resistance:</b> Strongest
predictor of multi-drug resistance - often indicates multi-class resistance patterns</li>
                                  <li><b style='color:#6c5ce7;'>Season & Location:</b> Seasonal peaks
and regional resistance patterns significantly influence bacterial prevalence</li>
                                </ul>
                                <p style='color:#8899cc; font-style:italic;'>These findings are consistent
with published clinical literature on antimicrobial resistance patterns.</p>
                                </div>"))))
       )
     )
   )
  )
)

#
================================================================================
====
# SERVER
#
================================================================================
====
server <- function(input, output, session) {
  authenticated <- reactiveVal(FALSE)
  diag_done <- reactiveVal(FALSE)
  patient_selected <- reactiveVal(NULL)

  output$logged_in <- reactive({ authenticated() })
  output$show_results <- reactive({ diag_done() })
  outputOptions(output, "logged_in", suspendWhenHidden = FALSE)
  outputOptions(output, "show_results", suspendWhenHidden = FALSE)

  output$locked_overlay <- renderUI({
```

```
  if (!authenticated()) {
    div(class = "locked-overlay",
        div(class = "locked-message",
            h2("SMART M.Dr", style = "color:#a29bfe; margin-top:10px;"),
            p("Multi-Drug Resistant Bacteria Prediction", style = "color:#a29bfe;"),
            br(),
            textInput("overlay_user", NULL, placeholder = "Username", width = "80%"),
            passwordInput("overlay_pass", NULL, placeholder = "Password", width = "80%"),
            actionButton("overlay_login", "ACCESS SYSTEM", class = "btn-primary", width =
"80%"),
            br(), br(),

        )
    )
  }
 })

 observeEvent(input$overlay_login, {
  if (input$overlay_user == "medecin" && input$overlay_pass == "smartmdr2024") {
    authenticated(TRUE)
    showNotification("ACCESS GRANTED", type = "message")
  } else {
    showNotification("ACCESS DENIED", type = "error")
  }
 })

 observeEvent(input$btn_load_patient, {
  req(input$diag_cin)
  tryCatch({
    patient <- data[data$CIN == input$diag_cin, ]
    if (nrow(patient) == 0) {
      output$auto_load_status <- renderUI({ div(class = "text-warning", "Patient not found.") })
    } else {
      p <- patient[nrow(patient), ]
      updateNumericInput(session, "diag_age", value = p$Age)
      updateSelectInput(session, "diag_sexe", selected = p$Sexe)
      updateSelectInput(session, "diag_diabete", selected = as.character(p$Diabete))
      updateSelectInput(session, "diag_hosp", selected = as.character(p$Hospitalisation))
      if (!is.na(p$Date)) updateDateInput(session, "diag_date", value = p$Date)
      updateTextInput(session, "diag_strain", value = p$EspeceBacterienne)
      updateTextAreaInput(session, "diag_symptoms", value = p$Symptomes)
      ab_inputs <- c("d_ab1","d_ab2","d_ab3","d_ab6","d_ab7","d_ab11","d_ab15")
      ab_indices <- c(1,2,3,6,7,11,15)
      for(i in 1:length(ab_inputs)) {
```

```
        val <- tryCatch(as.numeric(p[1, ab_columns[ab_indices[i]]]), error = function(e) NA)
        if(!is.na(val)) updateSelectInput(session, ab_inputs[i], selected = ifelse(val == 1, "R", "S"))
       }
       output$auto_load_status <- renderUI({ div(class = "text-success", paste("Loaded:",
p$PatientName)) })
     }
   }, error = function(e) {
     output$auto_load_status <- renderUI({ div(class = "text-danger", "Error loading patient.") })
   })
 })

 output$plot_mdr <- renderPlotly({
   mdr_df <- data.frame(Status = c("MDR", "Non-MDR"), Count = c(sum(data$MDR_Flag == 1),
sum(data$MDR_Flag == 0)))
   gg <- ggplot(mdr_df, aes(x = Status, y = Count, fill = Status)) +
     geom_bar(stat = "identity", width = 0.5, alpha = 0.9) +
     scale_fill_manual(values = c("MDR" = "#fd79a8", "Non-MDR" = "#00cec9")) +
     labs(x = "", y = "", title = paste("MDR Rate:", mdr_rate, "%")) +
     theme_minimal(base_size = 14) + theme(legend.position = "none")
   ggplotly(gg)
 })

 output$plot_resistance <- renderPlotly({
   resist_rates <- sapply(ab_columns, function(col) mean(data[[col]]) * 100)
   resist_df <- data.frame(Antibiotic = names(resist_rates), Rate = round(resist_rates, 1))
   ab_short <-
c("Amox/Amp","Amox/Clav","Cefazolin","Cefoxitin","Ceftriaxone","Imipenem","Gentamicin","Ami
kacin","Nalidixic","Ofloxacin","Ciproflox","Chloramph","Trim/Sulfa","Nitrofur","Colistin")
   resist_df$ShortName <- ab_short[1:nrow(resist_df)]
   gg <- ggplot(resist_df, aes(x = reorder(ShortName, Rate), y = Rate, fill = Rate)) +
     geom_bar(stat = "identity", alpha = 0.9) + scale_fill_viridis(option = "D") + coord_flip() +
labs(x = "", y = "") +
     theme_minimal(base_size = 12) + theme(legend.position = "none")
   ggplotly(gg)
 })

 output$table_bacteria_stats <- renderDT({
   datatable(bacteria_stats, options = list(pageLength = 8, scrollX = TRUE), rownames =
FALSE) %>%
     formatStyle("MDR_Pct", background = styleColorBar(range(bacteria_stats$MDR_Pct),
"#fd79a8"), backgroundSize = "100% 85%", backgroundRepeat = "no-repeat",
backgroundPosition = "center")
 })
```

```
  observeEvent(input$btn_search, {
    req(input$search_id)
    tryCatch({
      if (input$search_type == "id") results <- data[grepl(input$search_id, data$CIN, ignore.case =
TRUE), ]
      else if (input$search_type == "name") results <- data[grepl(input$search_id,
data$PatientName, ignore.case = TRUE), ]
      else results <- data[grepl(input$search_id, data$EspeceBacterienne, ignore.case = TRUE), ]
      if (nrow(results) == 0) {
        output$search_result <- renderUI({ div(class = "text-danger", "No results found.") })
        output$table_search_results <- renderDT({ NULL })
        output$patient_full_profile <- renderUI({ NULL })
      } else {
        output$search_result <- renderUI({ div(class = "text-success", paste("Found:",
nrow(results), "patient(s)")) })
        output$table_search_results <- renderDT({
          datatable(results[,
c("CIN","PatientName","Age","Sexe","EspeceBacterienne","MDR_Flag","Date")],
                    options = list(pageLength = 8, scrollX = TRUE), selection = "single", rownames =
FALSE) %>%
            formatStyle("MDR_Flag", backgroundColor = styleEqual(c(0,1),
c("#00cec940","#fd79a840")))
        })
        observeEvent(input$table_search_results_rows_selected, {
          idx <- input$table_search_results_rows_selected
          if (!is.null(idx)) patient_selected(results[idx, ])
        })
      }
    }, error = function(e) {
      output$search_result <- renderUI({ div(class = "text-danger", "Search error.") })
    })
  })

  output$patient_full_profile <- renderUI({
    req(patient_selected())
    p <- patient_selected()
    fp <- tryCatch(as.numeric(p[, ab_columns]), error = function(e) rep(0, 15))
    resistant_abs <- ab_columns[fp == 1]
    susceptible_abs <- ab_columns[fp == 0]
    tagList(fluidRow(column(12,
                            box(title = paste("MEDICAL RECORD:", p$PatientName), solidHeader =
TRUE, width = NULL,
                                fluidRow(
                                  column(3, div(style = "text-align:center; padding:20px;",
```

```
                                    div(style = "font-size:60px;", ifelse(p$MDR_Flag == 1,
"[BIOHAZARD]", "[OK]")),
                                    h3(ifelse(p$MDR_Flag == 1, "MDR POSITIVE", "Non-MDR"),
style = paste0("color:", ifelse(p$MDR_Flag == 1, "#fd79a8", "#00cec9"), ";"))))),
                           column(9,
                                fluidRow(column(3, p(strong("ID:"), p$CIN)), column(3,
p(strong("Age:"), p$Age)), column(3, p(strong("Bacteria:"), p$EspeceBacterienne)), column(3,
p(strong("Date:"), format(p$Date, "%d/%m/%Y")))),
                                hr(),
                                fluidRow(column(6, h4("RESISTANT TO:", style = "color:#fd79a8;"),
tags$ul(lapply(resistant_abs, tags$li))),
                                         column(6, h4("SUSCEPTIBLE TO:", style = "color:#00cec9;"),
tags$ul(lapply(susceptible_abs, tags$li))))))
                         ))))
  })

  observeEvent(input$btn_diagnose, {
    ab_map <- c("d_ab1" = 1, "d_ab2" = 2, "d_ab3" = 3, "d_ab6" = 6, "d_ab7" = 7, "d_ab11" = 11,
"d_ab15" = 15)
    resistance_vec <- rep(NA, 15)
    for (nm in names(ab_map)) {
      idx <- ab_map[nm]
      val <- input[[nm]]
      resistance_vec[idx] <- ifelse(val %in% c("R","I","Intermediate"), 1, ifelse(val == "S", 0, NA))
    }
    famil_count <- sum(sapply(antibiotic_families, function(fam) {
      any(resistance_vec[which(ab_columns %in% fam)] == 1, na.rm = TRUE)
    }))
    mdr_flag <- famil_count >= 3
    pressure <- evolutionary_pressure(input$diag_age, as.numeric(input$diag_diabete),
as.numeric(input$diag_hosp), as.numeric(input$diag_smoker))
    ab_reco <- recommend_antibiotics(resistance_vec)
    season <- get_season(input$diag_date)
    location <- input$diag_location
    season_bacteria <- season_bacteria_map[[season]]
    location_diseases <- location_disease_map[[location]]
    if(is.null(location_diseases)) location_diseases <- c("Various infections")

    output$res_mdr <- renderValueBox({
      valueBox(ifelse(mdr_flag, "MDR POSITIVE", "Non-MDR"), paste(famil_count, "/ 9 classes"),
               color = ifelse(mdr_flag, "red", "green"))
    })
    output$res_pressure <- renderValueBox({
      valueBox(pressure$level, paste("Score:", pressure$score), color = pressure$color)
```

```
  })
  output$res_ab_count <- renderValueBox({
    rcount <- sum(resistance_vec == 1, na.rm = TRUE)
    valueBox(paste(rcount, "/", sum(!is.na(resistance_vec))), "Resistant Antibiotics",
             color = ifelse(rcount > 5, "red", "yellow"))
  })
  output$res_bacteria <- renderValueBox({
    strain <- if (nchar(input$diag_strain) > 0) input$diag_strain else "Multiple"
    valueBox(strain, "Target Bacteria", color = "blue")
  })

  output$seasonal_context <- renderUI({
    tagList(
      h4(paste("SEASON:", season, "| LOCATION:", location), style = "color:#a29bfe;"),
      h5(paste("Common Bacteria in", season, ":"), style = "color:#fd79a8;"),
      tags$ul(lapply(season_bacteria, tags$li)),
      h5(paste("Common Diseases in", location, ":"), style = "color:#00cec9;"),
      tags$ul(lapply(location_diseases, tags$li))
    )
  })

  output$plot_effectiveness <- renderPlotly({
    gg <- ggplot(ab_reco, aes(x = reorder(ShortName, Effectiveness), y = Effectiveness,
                              fill = Effectiveness, text = paste(Antibiotic, "-", Effectiveness, "%"))) +
      geom_bar(stat = "identity", alpha = 0.9) + scale_fill_gradient(low = "#fd79a8", high =
"#00cec9", limits = c(0, 100)) +
      coord_flip() + labs(x = "", y = "") + theme_minimal(base_size = 13) + theme(legend.position
= "none")
    ggplotly(gg, tooltip = "text")
  })

  output$table_recommendations <- renderDT({
    datatable(ab_reco[, c("Antibiotic", "Class", "Effectiveness", "Status")], options =
list(pageLength = 15), rownames = FALSE) %>%
      formatStyle("Effectiveness", background = styleColorBar(c(0, 100), "#00cec9"),
backgroundSize = "100% 85%", backgroundRepeat = "no-repeat", backgroundPosition =
"center") %>%
      formatStyle("Status", backgroundColor =
styleEqual(c("Resistant","Susceptible","Intermediate"),
c("#fd79a840","#00cec940","#fdcb6e40")))
  })

  diag_done(TRUE)
  showNotification("DIAGNOSIS COMPLETE", type = "message")
```

```
})

observeEvent(input$btn_add_patient, {
  req(input$new_cin, input$new_name, input$new_strain)
  tryCatch({
    new_row <- data[1, ]
    new_row$patient_id <- input$new_cin
    new_row$patient_name <- input$new_name
    new_row$age <- input$new_age
    new_row$gender <- input$new_sexe
    new_row$diabetes <- input$new_diabete
    new_row$hospital_before <- input$new_hosp
    new_row$strain <- input$new_strain
    new_row$collection_date <- as.character(input$new_date)
    new_row$notes <- input$new_symptoms
    new_row$address <- input$new_location
    ab_map2 <- c("n_ab1"=1,"n_ab2"=2,"n_ab6"=6,"n_ab7"=7,"n_ab11"=11,"n_ab15"=15)
    for (nm in names(ab_map2)) {
      idx <- ab_map2[nm]
      new_row[1, ab_columns[idx]] <- input[[nm]]
    }
    for (col in ab_columns) {
      new_row[1, col] <- ifelse(new_row[1, col] %in% c("R","I"), 1, 0)
    }
    new_row$Diabete <- ifelse(new_row$diabetes == "Yes", 1, 0)
    new_row$Hospitalisation <- ifelse(new_row$hospital_before == "Yes", 1, 0)
    new_row$Age <- as.numeric(new_row$age)
    new_row$CIN <- new_row$patient_id
    new_row$PatientName <- new_row$patient_name
    new_row$EspeceBacterienne <- new_row$strain
    new_row$Date <- as.Date(new_row$collection_date)
    new_row$Sexe <- new_row$gender
    new_row$Localisation <- "Tunis"
    new_row$MDR_Flag <- (sum(sapply(antibiotic_families, function(fam) {
      any(new_row[1, fam] == 1)
    })) >= 3) * 1
    data <<- rbind(data, new_row)
    write.csv(data, "9ALB1.csv", row.names = FALSE)
    output$add_patient_status <- renderUI({
      div(class = "text-success", style = "font-size:16px;", "PATIENT ADDED & SAVED")
    })
    showNotification("PATIENT ADDED", type = "message")
    updateTextInput(session, "new_cin", value = "")
    updateTextInput(session, "new_name", value = "")
```

```
    }, error = function(e) {
      output$add_patient_status <- renderUI({
        div(class = "text-danger", "Error adding patient.")
      })
    })
  })
}

shinyApp(ui = ui, server = server)
```